



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Desarrollo de Aplicaciones Web	Unidad 1	Tema: La estandarización de la comunicación en el desarrollo de software.
	Semana 2	Tutor: Ing. Victoria Castro

Introducción:

Conventional Commits es una convención ligera sobre los mensajes de commit que proporciona un conjunto de reglas para crear un historial de mensajes explícito.

El Origen

Esta especificación nació de las entrañas de los equipos de ingeniería de Google, específicamente dentro del proyecto Angular. Los desarrolladores necesitaban una forma de que las máquinas pudieran "leer" el historial de cambios para automatizar tareas tediosas.

A partir de estas directrices de Angular, figuras como Ben Coe (fundador de standard-version) y la comunidad de Conventional Changelog formalizaron la especificación v1.0.0, convirtiéndola en el estándar de oro de la industria moderna.

Sección 1:

Importancia en la Ingeniería de Software

¿Por qué obligar a un desarrollador a escribir de una forma específica?

No es por estética, es por rigor técnico:

1. **Versionado Semántico Automático (SemVer):** Al identificar si un commit es un fix o un feat, las herramientas de CI/CD pueden decidir si la nueva versión del software debe ser un parche (0.0.x) o una versión menor (0.x.0) sin intervención humana.
2. **Generación de Changelogs:** Permite crear documentos de "Novedades" (RELEASES) instantáneos, agrupando cambios por categorías para los usuarios finales.
3. **Comunicación Asíncrona:** En equipos distribuidos, el mensaje de commit es la documentación primaria. Un historial limpio permite entender el porqué de una decisión técnica meses después de que se tomó.
4. **Gobernanza del Código:** Facilita la búsqueda de errores (regresiones) mediante herramientas como git bisect y filtros de búsqueda avanzados.

Sección 2:

Estructura del mensaje

<tipo>[ámbito opcional]: <descripción>
[cuerpo opcional]

Tipos de Identificadores

- **feat:** Una nueva funcionalidad (corresponde a una versión MINOR).
- **fix:** Corrección de un error (corresponde a una versión PATCH).
- **docs:** Cambios en la documentación.
- **style:** Cambios que no afectan el significado del código (espaciado, formato).
- **refactor:** Cambio de código que no corrige un error ni añade una funcionalidad.
- **perf:** Mejora de rendimiento.
- **test:** Adición o corrección de pruebas.
- **chore:** Cambios en el proceso de construcción o herramientas auxiliares.

Actividad

¡Hola a todos! Tras revisar nuestro material de lectura sobre el **La estandarización de la comunicación en el desarrollo de software**, es momento de preguntarnos lo siguiente:

¿Qué es SemVer y para que se utiliza?

Te invito a compartir tu respuesta en el **Foro de discusión de la Unidad**, espero leer tu respuesta.

Bibliografía y Recursos

Para una comprensión profunda y técnica, se recomiendan las siguientes fuentes:

- [ConventionalCommits.org](https://conventionalcommits.org): Especificación oficial completa traducida al español.
- [Semantic Versioning \(SemVer\)](https://semver.org): El sistema de numeración X.Y.Z en el que se basa esta convención.
- [Angular Contributor Guide](https://angular.io/guide/contributor): El documento histórico que dio origen a la convención.
- [Commitlint Documentation](https://commitlint.js.org): Herramienta líder para asegurar que el equipo cumpla con este estándar automáticamente

